

mutation-kill-matrix

BOB-102 round-13 — mutation kill matrix

Every mutation was applied to the REAL analyzer, its application PROVEN by an md5 delta plus the changed line, the full 197-assertion harness run against it, and the analyzer restored from a pristine copy afterwards — with the restore verified by md5 (§11.4.84 zero residue). A mutation whose md5 did not move is reported as NOT-APPLIED rather than as a survivor.

Analyzer under test: md5 b45d9647a42cccae205f56886690d74c

Round-12 SURVIVORS — all three now killed

MUT-1-widen-MUX_ANY exclusivity-fuzz	md5=b7175701 KILLED	by: r11-mux-
MUT-4-ruleD-drop-newline newline	md5=b99995b2 KILLED	by: r13-ruleD-
MUT-5-delete-group-poison no-prefix r13-poison-label r13-poison-no-prefix	md5=d67eebe3 KILLED	by: r13-bypass-

Round-12 DO-NOT-REGRESS — all four still killed

MUT-2-immediate-next-line comment-gap gomux-split-blank-gap gomux-split-comment-gap r13-eng-split-comment	md5=72f623df KILLED	by: gin-split-
MUT-3-disable-RULE-S trace-authed gomux-method-funcresult	md5=a677abf3 KILLED	by: fastapi-
MUT-8-first-brace-wins functype gomux-body-typedeccl r13-case-functype r13-ruleD-newline r13-semi-functype	md5=28aa31f6 KILLED	by: gomux-body-

MUT-7-demote-ws-refusal md5=a3cb6089 KILLED by: gomux-ws-selector r11-before-dot-label r11-ws-label

Round-12 EXPECTED SURVIVOR — still survives, still honestly UNPINNED

MUT-6-drop-dead-dotted-term md5=cec2d94e SURVIVED by: (none)

The `MUX\_DOTTED\_OWNER` term in `\_named` is dead by preemption – the dotted refusal above it returns on `.search()`, truthy for every line `.findall()` would count – so it is semantically unreachable and NO fixture can pin it. Kept as correct arithmetic for a future reordering, labelled UNPINNED in-source. Its survival is the expected verdict, not a gap.

Round-13 NEW — each killed by its own fixture (diagonal)

M13-1-engine-rigid-spelling md5=8ea6c0be KILLED by: r13-eng-ws-around r13-eng-ws-paren

M13-2-drop-engine-split md5=c0864e05 KILLED by: r13-eng-alias-split r13-eng-both-split r13-eng-split-comment r13-eng-split-newline

M13-3-drop-alias-resolution md5=c0903f06 KILLED by: r13-eng-alias-split r13-eng-alias-two r13-eng-dot-import r13-eng-dot-import-label

M13-4-drop-dot-import-refusal md5=8c9171d0 KILLED by: r13-eng-dot-import r13-eng-dot-import-label

M13-5-drop-unmod-split md5=4f0d1fc5 KILLED by: r13-unmod-Any r13-unmod-Handle r13-unmod-Match r13-unmod-NoMethod r13-unmod-NoRoute r13-unmod-Static r13-unmod-StaticFile r13-unmod-StaticFileFS r13-unmod-StaticFS r13-unmod-ws

M13-6-ruleD-drop-colon md5=4b5b2ac3 KILLED by: r13-case-functype

M13-7-drop-float-discriminator md5=ae5040d0 KILLED by: r13-float-literal

M13-8-poison-dot-line-only md5=58f4b6ee KILLED by: r13-bypass-no-prefix

M13-9-drop-dropped-label md5=07480834 KILLED by: r13-poison-label

M13-10-crash-exits-1 md5=1c18cc97 KILLED by: r13-crash-is-error

Note on M13-4 — a FAIL-bluff found in this round's OWN fixture

M13-4-drop-dot-import-refusal SURVIVED the first battery. The mutation made the engine-regex builder raise a `TypeError`; Python exits **1** on an unhandled exception, and 1 is this analyzer's FAIL code, so the fixture's `expected=1` was satisfied by a crash. The fixture could not tell “refused correctly” from “the analyzer is in pieces” — a §11.4.1 FAIL-bluff living in the test, not the code.

Both halves are fixed: the analyzer now traps and exits **2** (the wrapper's `ERROR` class, never mistaken for a verdict about a route), and the fixture gained a label assertion. `r13-crash-is-error` pins the new behaviour with an injected crash on a scratch copy plus an uninjected control needle.